

Containerlab Node Manager — Student Quick Start

Guide revision 1 · tested with manager release 1.30.35 · September 2026

This guide walks you through the manager from the browser: deploying a lab, working on its devices from the command line, and saving your device configurations to Git so they are not lost even after the lab itself is destroyed. Pick the path that matches what you were given and follow it step by step; each step says what to do and what you should see.

Your starting point	Follow
I was given a lab by my instructor	Scenario A
I want to build my own lab and keep it in Git	Scenario B

Before you start	1
Scenario A: use a lab your instructor gave you	3
Scenario B: build your own lab and keep it in Git	12
Reference	19
What is saved where	19
Stop, Destroy, Remove	19
Troubleshooting	20
Next session in 60 seconds	20

Before you start

What you have been given

Item	What it is
Manager address	<code>http://192.168.132.132:8081</code> — open it in a browser; there is no sign-in
Your GitHub account	The one the lab VM is signed in to; in this guide it is <code>pruger-dev</code> — wherever you see <code>pruger-dev</code> in an address, read your own account name
A terminal on the lab VM	A shell as your ordinary VM account (in this guide <code>clablm</code>), used only in the few steps marked <i>Where to run it: on the lab VM</i>
Instructor materials	The <code>link-basics</code> lab is already staged on the lab VM, with its starting point, finished version and a broken version saved in the course repository (Scenario A)
Course repository	<code>https://github.com/pruger-dev/netlab-course</code> — the instructor's copy; you read it, you never push to it

Item	What it is
Your own repository	Scenario A: made on GitHub from the course repository with Use this template > Create a new repository (name it, private is fine) before you begin — https://github.com/pruger-dev/netlab-course-student.git (your account, your copy's name). Scenario B: a brand-new, empty repository you create in step B5

Everything below happens in one of three places: **your computer** (the browser you are reading this in, and any terminal you run on your own machine), **the lab VM** (the server that actually runs the lab's devices, and where the manager itself lives), and **your online repository** (your account on GitHub, out on the internet). A file can exist in one of these and not the others until you deliberately move it — keeping track of which place you are looking at is most of what this guide teaches.

A few Git words come up along the way. A **repository** is a project's saved history, kept on GitHub and also as a copy elsewhere; a **commit** is one saved snapshot of some files with a short message; to **push** — this guide also calls it **upload** — is to send your commits to the copy on GitHub; and to **clone** is to copy a whole repository from GitHub onto another machine. Each word is used again later without repeating its definition.

Scenario A: use a lab your instructor gave you

In this scenario you deploy a lab your instructor already prepared, look at it from the command line, make a small change, save your work to Git, and practice recovering from a fault — all using `link-basics`, two routers, `r1` and `r2`, joined by one link.

Save progress saves, for each device, two files — the readable configuration (`.cfg`) and the copy Apply to running lab... uses (`.eoscfg`) — plus a `manifest.json` that lists them. It does not save the topology file, the map or a README. For `link-basics` the topology file is already staged on the lab VM and included in the course repository, so you never need to save it yourself in this scenario.

A1 Open the manager

Action In a browser on your computer, go to `http://192.168.132.132:8081`.

Expected result Home shows a **Deploy** box, a **Build** box, a **Manager** ▼ menu and a lab list. Before anything is deployed the list reads **No labs yet**.

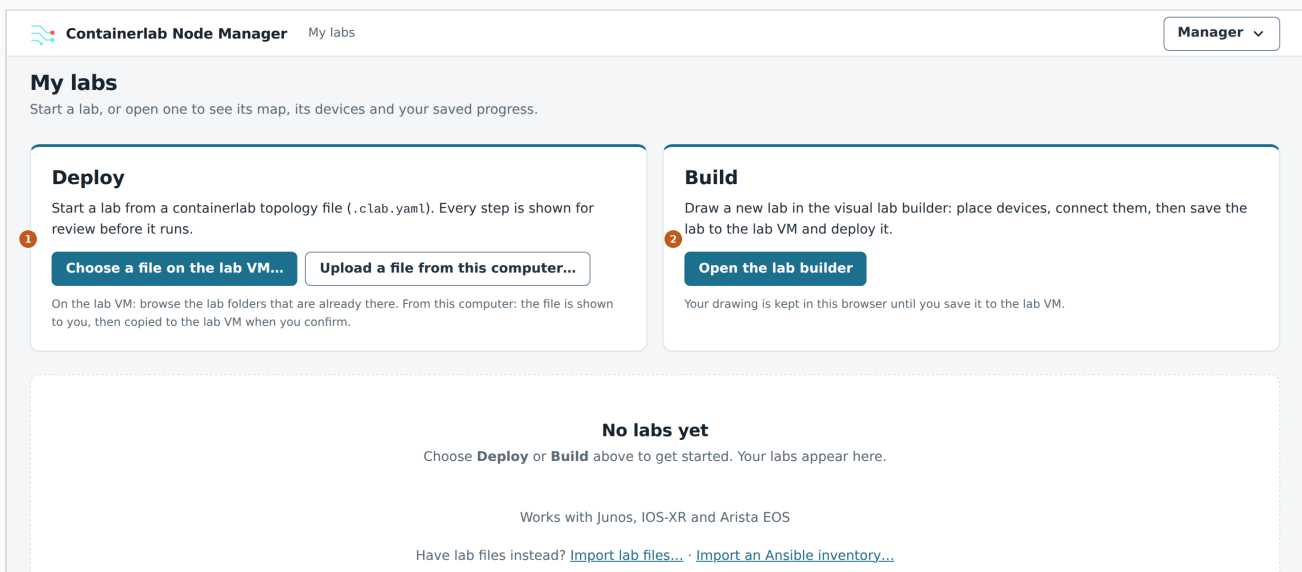


Figure A.1 — Home page before any lab is deployed.

A2 Deploy the lab

Action In the Deploy box, click **Choose a file on the lab VM...**. The **Deploy a new lab** dialog opens on the lab folders on the VM; open `/srv/containerlab-node-manager/projects`, then the `link-basics` folder inside it, and click ◇ **link-basics.clab.yml**.

Expected result A **Topology file** dialog opens showing the file location on the VM and a read-only preview of the topology (YAML), with buttons **Preview topology**, **Edit visually...**, **Add to My labs without starting** and **Deploy lab**.

Action Click **Deploy lab**.

Expected result A review titled **Start link-basics?** opens: "Creates and starts the devices in this topology. Nothing is deleted; device logins open as the devices boot." Confirm with **Start lab**.

If not If `link-basics` does not appear in the folder browser, ask your instructor to confirm it is staged on the lab VM — the browser only shows lab folders the manager trusts.

Figure A.2a — The Topology file dialog for `link-basics.clab.yml`.

Figure A.2b — The Start link-basics? review.

A3 Wait for the devices to become ready

Action Stay on the lab page while `r1` and `r2` start.

Expected result A running container is not yet a router that accepts a login. Right after you confirm, the header reads **Starting lab** and **0 of 2 devices ready**, and each device's row reads **Unavailable** for a moment, then **Starting**, then **Ready** once the manager can log in and get an answer from it — about a minute on this VM; up to a few minutes on a slower one. Wait until the header reads **2 of 2 devices ready**.

If not A pill reading **Needs credentials** means the device is up but the manager could not log in — check the login with your instructor. **Needs attention** after several failed attempts offers **Test login now** to retry once the problem is fixed.

Figure A.3 — Both devices read Ready.

A4 Look around from the command line

Action On the Devices tab, click **Open CLI** next to `r1`. It opens a terminal in a new browser tab; wait until its status reads **Connected**, then at the `r1>` prompt run:

```
r1>show version
r1>show interfaces status
r1>show ip interface brief
```

Expected result `show version` reports the platform; `show interfaces status` shows `Et1` as connected, since the link between `r1` and `r2` is physically up; `show ip interface brief` lists only `Management0` — `Ethernet1` has no address yet, so EOS does not list it there. That is expected: the lab starts with the link wired but nothing configured on it.

```
Last login: Tue Sep 22 17:27:27 2026 from 172.20.20.1
r1>show version
Arista cEOSLab

r1>show interfaces status
Port      Name      Status      Vlan      Duplex Speed  Type              Flags Encapsulation
Et1       Name      connected    1         full   1G     EbraTestPhyPort
Ma0       connected routed      a-full a-1G    10/100/1000
```

Figure A.4 — `r1`'s terminal after the show commands.

A5 Connect your repository

Action Make sure you already have your own copy of the course repository — made on GitHub with **Use this template > Create a new repository** before you begin. Open the **Progress** tab: with nothing connected yet it shows **Choose where to save your progress**; click **Connect a repository by URL**. Paste `https://github.com/pruger-dev/netlab-course-student.git` (your own copy's address, not the instructor's), set the folder to `link-basics/work`, tick the acknowledgement that complete device configurations will be saved and uploaded, and click **Connect repository**.

Expected result The destination line reads exactly "link-basics saves to netlab-course-student > link-basics/work > latest/". **Saved versions** now also shows a group, **Instructor and reference versions** — "Versions your instructor put in the repository appear here. Apply one to load it onto your running devices; the current configuration is backed up first." — listing three rows (in any order), each with 5 files and **View**, **Compare with my latest save** and **Apply to running lab...: Starting state** (`link-basics/reference/start/latest`), **Final state (instructor)** (`link-basics/reference/solution/latest`) and **Troubleshooting scenario 01** (`link-basics/reference/broken-01/latest`).

If not If the manager reports it cannot reach or push to the repository, check that you made your own copy of the course repository with **Use this template > Create a new repository** on GitHub first — the URL above must point at your copy, not the instructor's.

Connect a repository by URL ✕

Paste the repository's HTTPS address from GitHub (Code > HTTPS). The lab VM's existing GitHub login is used — you are never asked for a password or token.

1 **Repository URL**

2 **Folder for this lab** optional

3 ☒ I understand that complete device configurations — including any passwords or keys they contain — will be saved to this repository and uploaded to its online copy.

4 **Cancel** **Connect repository**

Figure A.5a — The Connect a repository by URL dialog, filled in.

1 **Saving to netlab-course-student > link-basics/work** **Save progress** **Create checkpoint...** **More** ▾

Not saved yet · Save progress creates a snapshot you can return to later.

Saved versions
Versions of this lab you can return to. Save progress creates a new one.

Latest
You haven't saved this lab yet. Save progress creates a configuration snapshot you can return to later.

Checkpoints
No checkpoints yet. Create a checkpoint when you reach an important milestone.

Instructor and reference versions
Versions your instructor put in the repository appear here. Apply one to load it onto your running devices; the current configuration is backed up first.

Version	Files	View	Compare with my latest save	Apply to running lab...
Troubleshooting scenario 01 link-basics/reference/broken-01/latest	5 files	View	Compare with my latest save	Apply to running lab...
Final state (instructor) link-basics/reference/solution/latest	5 files	View	Compare with my latest save	Apply to running lab...
Starting state link-basics/reference/start/latest	5 files	View	Compare with my latest save	Apply to running lab...

2

Figure A.5b — The Progress tab once the repository is connected.

A6 Load the starting configuration

Action On the Progress tab, find **Starting state** and click its **Apply to running lab...**

Expected result A review titled **Replace running configuration** opens, its source line reading "Source: Starting state · netlab-course-student > link-basics/reference/start/latest · saved ... · <commit>". Each device is listed, for example "r1 EOS 6 differences from the running configuration". Tick "I understand the running configuration on the selected devices will be replaced." and click **Replace configurations** — if a device cannot be reached again within five minutes, it undoes the change on its own.

Expected result Within about a dozen seconds each device reports "Configuration replaced and verified against the saved desired state." Go back to r1's terminal tab from A4 (still open) and run `ping 10.0.0.2` — it succeeds, since `start` addresses the link `10.0.0.1/30` on r1 and `10.0.0.2/30` on r2.

Replace running configuration ×

Lab: **link-basics**

1 Source: **Starting state** · **netlab-course-student** · **link-basics/reference/start/latest** · saved 35 minutes ago · 089a35aeb8

Current configurations are backed up first. The devices are not rebooted.

Devices

- ☒ **r1** EOS 6 differences from the running configuration
- ☒ **r2** EOS 6 differences from the running configuration

- Each device's current configuration is backed up first. A device whose backup fails is left unchanged.
- The devices are not rebooted. The device itself checks the new configuration when it is activated, and the change is undone on its own if the device cannot be reached again within 5 minutes.
- Devices that are skipped above are not touched.

► **Advanced options**

2 ☐ I understand the running configuration on the selected devices will be replaced.

Cancel
3 **Replace configurations**

Figure A.6 — Replace running configuration, reviewing Starting state.

A7 Make a change on r1**Action** On r1's CLI:

```

r1>enable
r1#configure terminal
r1(config)#interface Loopback0
r1(config-if-Lo0)#description r1 loopback
r1(config-if-Lo0)#ip address 10.255.0.1/32
r1(config-if-Lo0)#end
r1#write memory
r1#show ip interface brief

```

On EOS a configuration command is active the moment you enter it; `write memory` only keeps it across a device restart, and **Save progress** in the next step captures the running configuration either way.

Expected result `show ip interface brief` now also lists `Loopback0` with address `10.255.0.1/32`. `r2` is untouched for now — its own loopback comes in step A9.

```

Last login: Tue Sep 22 17:28:10 2026 from 172.20.20.1
r1>enable
r1#configure terminal
r1(config)#interface Loopback0
r1(config-if-Lo0)#description r1 loopback
r1(config-if-Lo0)#ip address 10.255.0.1/32
r1(config-if-Lo0)#end
r1#write memory
Copy completed successfully.
r1#show ip interface brief

```

Interface	IP Address	Status	Protocol	MTU	Address Owner
Ethernet1	10.0.0.1/30	up	up	1500	
Loopback0	10.255.0.1/32	up	up	65535	
Management0	172.20.20.11/24	up	up	1500	

```
r1#
```

Figure A.7 — Loopback0 added and read back on r1.

A8 Save your progress

Action Click **Save progress** in the lab header.

Expected result The manager captures both devices and opens **Review before uploading**: "What this save changed compared with the previous one. Configuration files may contain passwords or keys." and "This save is on the lab VM only. Nothing is uploaded to github.com unless you choose Upload these changes." Because this is the first save, every file is listed as added: r1.cfg added, r1.eoscfig added, r2.cfg added, r2.eoscfig added. **Upload these changes** sends it to your repository at once; **Not now — keep it on the VM** leaves it waiting under **Recent saves** instead. Click **Upload these changes**.

Expected result The Progress tab reads **Saved to Git just now**, and the **Latest** row reads "netlab-course-student > link-basics/work > latest · Saved just now · 5 files" — the four device files reviewed above, plus the manifest.json that lists them. On GitHub, link-basics/work/latest/ now holds exactly those five files.

Review before uploading

What this save changed compared with the previous one. Configuration files may contain passwords or keys.

This save is on the lab VM only. Nothing is uploaded to github.com unless you choose **Upload these changes**.

1 Details

r1.cfg
added

r1.eoscfig
added

r2.cfg
added

r2.eoscfig
added

2
3

Open the full saved version

Not now — keep it on the VM

Upload these changes

Figure A.8 — Review before uploading, everything added.

A9 Checkpoint, then add r2's loopback

Action Open the **Save progress** menu (▼) and choose **Create checkpoint....** Name it `loopback-added` and confirm; it opens the same **Review before uploading** — click **Upload these changes**.

Action Now give `r2` its own loopback: on `r2`'s CLI, run the same commands as in A7, using `description r2 loopback` and `ip address 10.255.0.2/32`, then `write memory`. Click **Save progress** again and upload that review too.

Expected result This second save updates the same `link-basics/work/latest` in place — it never creates a nested `latest/latest`. **Saved versions** now lists **Latest** ("Saved just now · 5 files") and, under **Checkpoints**, `loopback-added`. On GitHub, `link-basics/work/latest/r2.cfg` now contains `Loopback0`, while `link-basics/work/checkpoints/loopback-added/r2.cfg` does not — the checkpoint captured `r2` before its loopback existed.

Figure A.9a — Naming the checkpoint `loopback-added`.

Figure A.9b — Saved versions listing **Latest** and the new checkpoint.

A10 Break it, then recover

Action Find **Troubleshooting scenario 01** and click its **Apply to running lab...**, then confirm with **Replace configurations**.

Expected result Both devices are replaced. On `r1`'s CLI, the ping fails and `r1` shows its `Ethernet1` as `notconnect`: the link is down at `r2`'s end. See if you can spot why before you move on.

Action Recover by applying your own saved progress — this undoes the fault (the instructor's broken state shut down `r2`'s `Ethernet1`): find **Latest** under **Saved versions** and click its **Apply to running lab...**, then **Replace configurations** again. Its source line names your own folder: "Source: work · netlab-course-student > link-basics/work/latest · ...".

Expected result ping `10.0.0.2` from `r1` succeeds again and both loopbacks are back. The *Save location* card still shows the lab saving to `link-basics/work` — applying a saved version never changes where the lab saves. Curious what changed since your checkpoint? The `loopback-added` row's **Compare with my latest save** opens a dialog titled **Compared with your latest save**.

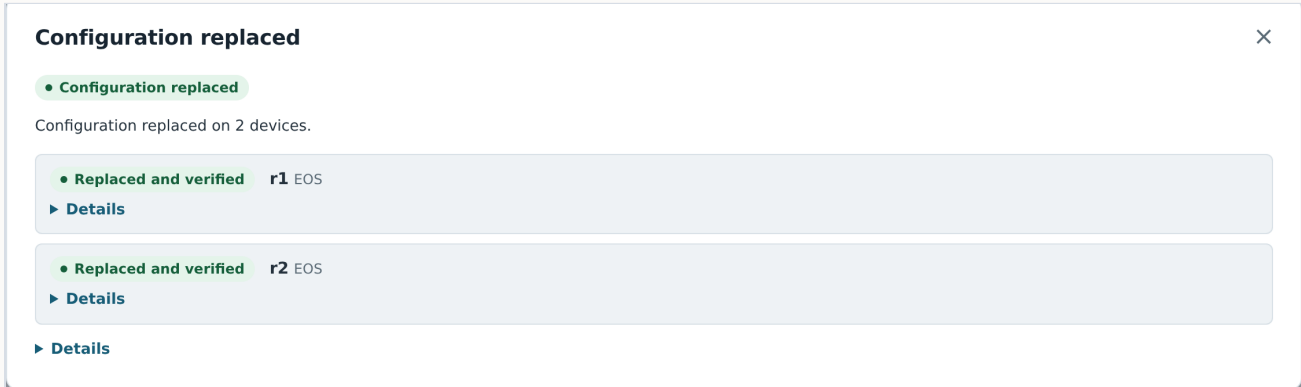


Figure A.10 — Your own Latest applied; the link is restored.

A11 Next time you open this lab

Action Close the browser, then open the manager again at `http://192.168.132.132:8081`.

Expected result The `link-basics` card on Home reads "link-basics · Running · 2 of 2 devices ready · Last saved N minutes ago · Deployed N minutes ago", with an **Open lab** button. Open it — the Progress tab still lists **Latest** and your `loopback-added` checkpoint, exactly as you left them.

Action Open **Lab actions** ▼ to see everything you can do with this lab — among the everyday actions are **Stop devices**, **Redeploy lab...**, **Destroy lab...** and **Remove from this manager...**; the rest of the menu is in the figure below.

Open **Destroy lab...** to see its review — "The running devices are removed from the VM. Configuration changes you have not saved are lost. Your saved progress, checkpoints and backups remain." with a **Save progress first** option — then click **Cancel**; there is no need to destroy this lab now. **Remove from this manager...** is even less drastic: "Nothing on the lab VM changes: the running devices and the topology files stay. Progress you saved to Git stays in the repository."

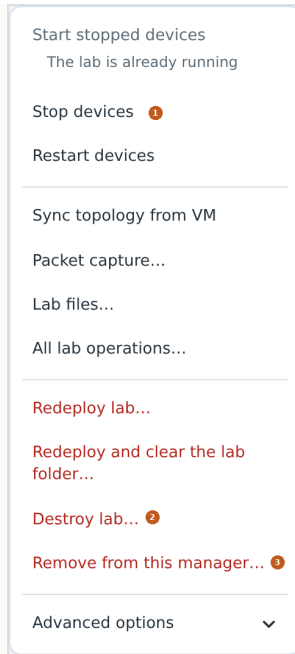


Figure A.11 — The Lab actions ▼ menu.

Checklist

- [] `link-basics` deployed and both devices read **Ready**
- [] Repository connected, saving to `link-basics/work`
- [] **Starting state** applied and the link pings
- [] A loopback added on `r1`, checkpointed, then a loopback added on `r2`
- [] Progress **Saved to Git just now**
- [] Checkpoint `loopback-added` created
- [] **Troubleshooting scenario 01** applied, then your own **Latest** applied to recover

Scenario B: build your own lab and keep it in Git

In this scenario you draw your own two-router lab in the visual builder, deploy it, configure it, and set up your own repository so your work survives even after the lab is destroyed and removed from the manager. The devices are `r1` and `r2`, joined by one link, both Arista cEOS.

Save progress saves, for each device, two files — the readable configuration (`.cfg`) and the copy Apply to running lab... uses (`.eoscfg`) — plus a `manifest.json` that lists them. It does not save the topology file, the map or a README. A lab definition and its saved device configurations are two different things — this scenario ends with both of them safely in Git, and step B7 is exactly the extra step that gets the topology and map there.

B1 Start a new lab in the builder

Action From Home, click **Open the lab builder** in the Build box. The builder page opens in the same tab and shows **Build a lab** with **New lab...** and **Open a draft...**; click **New lab...**. Fill in **Lab name** (`my-first-lab` — letters, digits, dot, dash and underscore only), **Start from** (**Two devices, one link**), **Device type for the starter** (**Arista cEOS · n24l/ceos:4.35.0F** — the image shown is only a suggestion, taken from labs already in My labs or a default) and **Lab folder on the VM** (`/srv/containerlab-node-manager/projects`). Click **Create draft**.

Expected result The editor opens with two devices, named `ceos1` and `ceos2` by default, already joined by one link. The status pill reads **Draft · kept in this browser only · not on the VM yet**.

Figure B.1 — The New lab dialog, filled in.

B2 Draw and check the topology

Action Right-click the first device and choose **Edit Node**. In the Node Editor panel (tabs Basic / Configuration / Runtime / Network / Advanced), set **Node Name** to `r1` and, on the Basic tab, **Image** `n24l/ceos` and **Version** `4.35.0F`; on the Network tab, set **Management IPv4** to `172.20.20.21`. Click **Apply**. Repeat for the second device: name it `r2`, the same image and

version, and Management IPv4 172.20.20.22 . A saved management address like this one is recommended because it applies again after a redeploy.

Changing something on the Network tab can reset the image back to its suggested name, so glance at the Basic tab again before you click **Apply**, and confirm with **View YAML** that each device reads the image your VM has (here n24l/ceos:4.35.0F). While you are there, right-click empty canvas and choose **Add Text** to note what the lab is, for example "My first lab: r1 eth1 - r2 eth1".

Expected result **View YAML** shows the topology: name: my-first-lab , both nodes with kind: arista_ceos , image: n24l/ceos:4.35.0F and their management addresses, and one link, r1:eth1 - r2:eth1 .

The screenshot shows the 'Node Editor' window with the 'BASIC' tab selected. The 'Node Parameters' section contains the following fields:

- Node Name:** r1
- Kind:** arista_ceos
- Image:** n24l/ceos
- Version:** 4.35.0F
- Icon:** Switch (with an 'EDIT' button)
- Label & Direction:**
 - Label Position:** Bottom
 - Label Text Direction:** Horizontal
 - Label Background:** # 000000
 - Transparent:** ☐

An 'APPLY' button is located at the bottom right of the editor.

Figure B.2a — The Node Editor, setting r1's name, image and version.

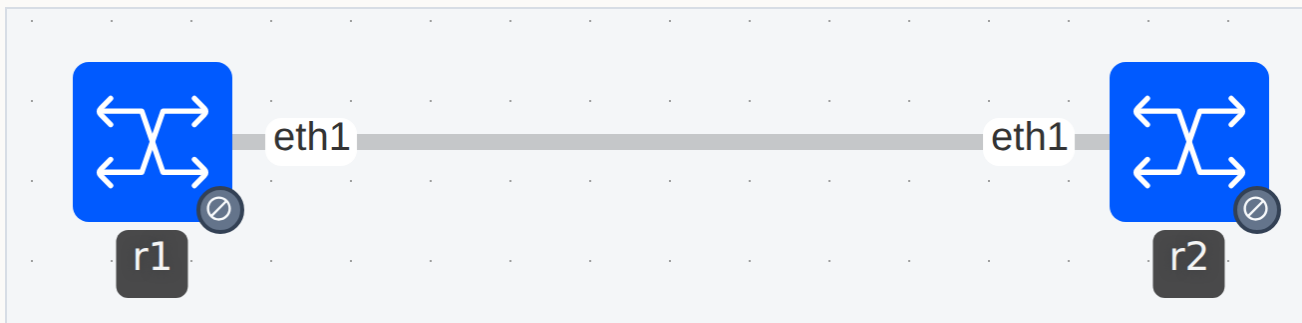


Figure B.2b — Finished canvas: r1 and r2 joined by one link.

B3 Save to the VM and deploy

Action Click **Save to the VM...** A review titled **Save my-first-lab to the VM?** shows the lab folder, /srv/containerlab-node-manager/projects/my-first-lab/my-first-lab.clab.yml , and the YAML that will be written; confirm with **Save lab**.

Expected result The result reads "✓ Save lab to the VM succeeded" and the status pill changes to **Saved on the VM**.

Action Click **Deploy or add this lab...**, then in the **Topology file** dialog click **Deploy lab**. A review titled **Start my-first-lab?** opens; confirm with **Start lab**.

Expected result The builder page itself does not navigate anywhere next — click ← **My labs** at the top left and open the `my-first-lab` card from Home to continue.

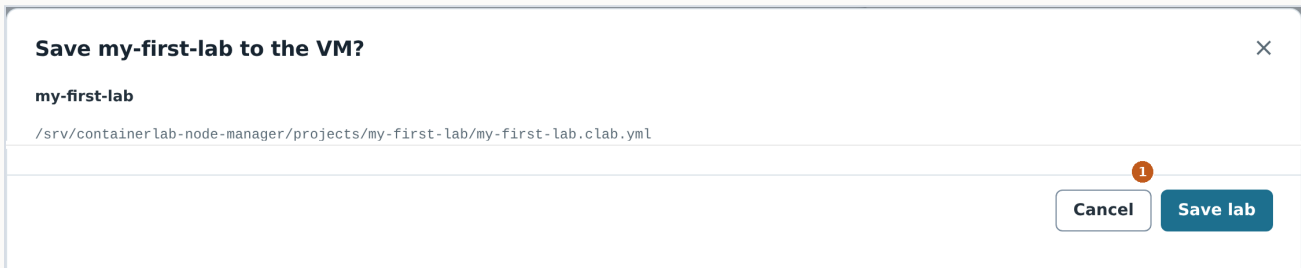


Figure B.3 — The review before the lab folder is written to the VM.

B4 Bring the link up

Action Wait for both devices to read **Ready** — about a minute on this VM; up to a few minutes on a slower one. Open a CLI on `r1` and run:

```
r1>enable
r1#configure terminal
r1(config)#ip routing
r1(config)#interface Ethernet1
r1(config-if-Et1)#no switchport
r1(config-if-Et1)#ip address 10.0.0.1/30
r1(config-if-Et1)#end
r1#write memory
```

`no switchport` matters here: without it, EOS keeps `Ethernet1` as a switched port and refuses the `ip address` that follows. On `r2`, run the same commands with `ip address 10.0.0.2/30`. From `r1`, run `ping 10.0.0.2`.

Expected result The ping succeeds.

```
Last login: Tue Sep 22 18:43:07 2026 from 172.20.20.1
r1>ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 72(100) bytes of data:
80 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.132 ms
80 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.007 ms
80 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.003 ms
80 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.005 ms
80 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.005 ms

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.003/0.030/0.132/0.050 ms, ipg/ewma 0.053/0.079 ms
r1>
```

Figure B.4 — Both devices Ready, link addressed and pinging.

B5 Create your own repository on GitHub

Where to run it On your computer.

Action On GitHub, choose **New repository**. This is created under the same GitHub account the lab VM is signed in to — `pruger-dev` in this guide's example, your own account for you. Name it `my-network-labs`, set it **Private**, tick **Add a README file**, and click **Create repository**. From a terminal with GitHub's command-line tool installed, the same repository can be created with:

```
$ gh repo create my-network-labs --private --add-readme
```

Copy its **Code** → **HTTPS** address for the next step.

Expected result The repository exists, private, on branch `main`, with a `README.md`.

B6 Connect the repository and save your progress

Action In the manager, open `my-first-lab` → **Progress** tab → **Connect a repository by URL**. Paste `https://github.com/pruger-dev/my-network-labs.git` (your account), set the folder to `my-first-lab`, tick the acknowledgement that complete device configurations will be saved and uploaded, and click **Connect repository**.

Expected result The destination line reads exactly "my-first-lab saves to my-network-labs > my-first-lab > latest/".

Action Click **Save progress. Review before uploading** lists everything as added — `r1.cfg` added, `r1.eoscfg` added, `r2.cfg` added, `r2.eoscfg` added, since this is the first save — then click **Upload these changes**.

Expected result The status reads **Saved to Git just now**. On GitHub, `my-first-lab/latest/` holds `manifest.json`, `r1.cfg`, `r2.cfg`, `r1.eoscfg` and `r2.eoscfg` — and no topology file: that still only exists on the lab VM until the next step.

Figure B.6 — Connecting `my-network-labs` with folder `my-first-lab`.

B7 Publish the topology files

Where to run it On the lab VM, as your ordinary account.

Action **Save progress** only uploads device configurations — the topology file and its map are still only on the lab VM. The folder `~/labs/my-network-labs` was created on the lab VM when you connected the repository in B6; the manager keeps its own copy of your repository there. Publish the topology files into that same checkout:

```
$ cd ~/labs/my-network-labs
$ mkdir -p my-first-lab
$ cp /srv/containerlab-node-manager/projects/my-first-lab/my-first-lab.clab.yml* my-first-lab/
```

Write `my-first-lab/README.md`, for example:

```
# my-first-lab
Two Arista cEOS routers, r1 and r2, joined by one link (r1 eth1 - r2 eth1).
Link addresses: 10.0.0.1/30 on r1, 10.0.0.2/30 on r2.
Saved progress lives in latest/ (and checkpoints/ for named milestones).
Topology: my-first-lab.clab.yml; map: my-first-lab.clab.yml.annotations.json.
```

Then commit and push:

```
$ git add my-first-lab
$ git commit -m "my-first-lab: topology and map"
$ git push
$ git status -sb
```

Expected result `git status -sb` reports `## main...origin/main` — clean, nothing left to push. On GitHub, `my-first-lab/` now holds `README.md`, `my-first-lab.clab.yml`, `my-first-lab.clab.yml.annotations.json` and `latest/`. The checkout must stay clean and pushed like this before the manager's next **Save progress**, or that save is refused.

```
B7 - owner terminal, lab VM (as clabllm)

$ cd /home/clabllm/labs/my-network-labs && mkdir -p my-first-lab && cp /srv/containerlab-node-manager/projects/my-first-lab/my-first-lab.clab.yml* my-first-lab/

$ cat > my-first-lab/README.md # 5-line description written above

$ git add my-first-lab

$ git commit -m "my-first-lab: topology and map"
[main 127bfe8] my-first-lab: topology and map
3 files changed, 65 insertions(+)
create mode 100644 my-first-lab/README.md
create mode 100644 my-first-lab/my-first-lab.clab.yml
create mode 100644 my-first-lab/my-first-lab.clab.yml.annotations.json

$ git push
To https://github.com/pruger-dev/my-network-labs.git
a6121e5..127bfe8 main -> main

$ git status -sb
## main...origin/main
```

Figure B.7 — Publishing the topology and map from the lab VM.

B8 Check GitHub

Where to run it On your computer.

Action Open `my-network-labs` on GitHub.

Expected result The repository shows a `my-first-lab` folder holding the topology file, its map, the README you wrote, and the `latest/` folder your save uploaded. Back in the manager, the same tree is one click away: **Progress** > **Saved versions** > **Browse the repository...** opens the repository browser on the lab's own folder and lists `README.md`, `my-first-lab.clab.yml`, `my-first-lab.clab.yml.annotations.json` and `latest/` side by side — the in-product view of exactly what GitHub holds.

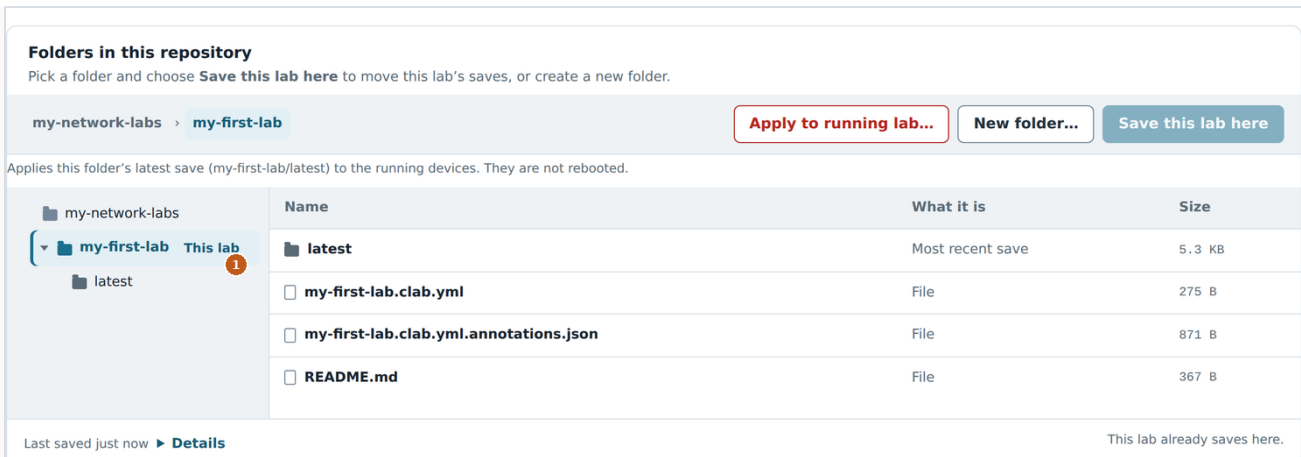


Figure B.8 — Browsing the repository from the Progress tab.

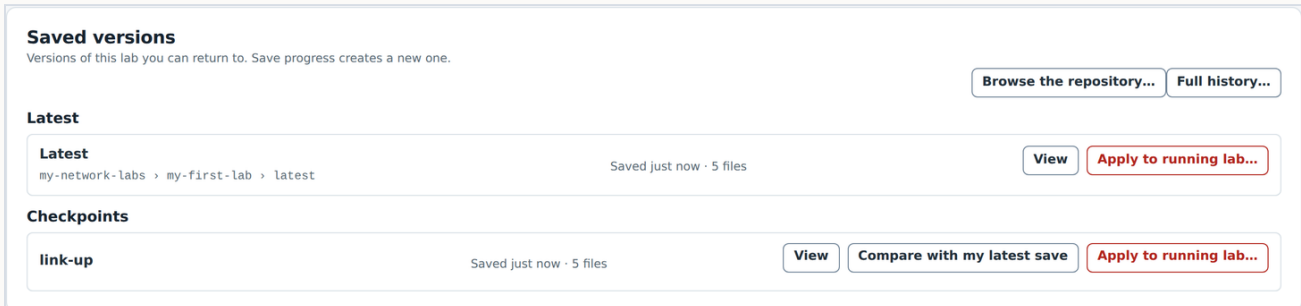
B9 Change, save, and checkpoint again

Action On `r2`'s CLI, add a `Loopback0` addressed `10.255.0.2/32`, the same way as in Scenario A, then write memory. Click **Save progress**.

Expected result This time **Review before uploading** lists only `r2.cfg` changed and `r2.eoscfg` changed — `r1` did not change. Click **Upload these changes**; the status reads **Saved to Git just now**.

Action Open the **Save progress** menu and choose **Create checkpoint...**, name it `link-up`, and upload that review too.

Expected result **Saved versions** lists **Latest** (`my-network-labs > my-first-lab > latest`) and, under **Checkpoints**, `link-up`. On GitHub, `my-first-lab/latest/r2.cfg` contains the new `Loopback0`, `my-first-lab/checkpoints/link-up/` exists, and there is still no nested `latest/latest` — the two saves simply added two more commits ("Save my-first-lab progress") above "my-first-lab: topology and map".

Figure B.9 — Checkpoint `link-up` alongside `Latest`.**B10 Prove it: destroy, remove, and rebuild from Git**

Action From **Lab actions** ▼, choose **Destroy lab...**. The review, titled **Destroy my-first-lab?**, warns the same way as in Scenario A; confirm with **Destroy lab**.

Expected result The lab header reads **Stopped** — the devices are gone.

Action From **Lab actions** ▼, choose **Remove from this manager...** and confirm with **Remove lab**. Its checkbox **Don't offer this lab for import again** can stay ticked or not — either is fine

here. This step only removes the lab from this manager; it never touches your repository or the topology file you published in B7.

Where to run it On the lab VM, as your ordinary account. Clone your repository into a trusted lab folder, since the manager's deploy browser only shows folders it trusts:

```
$ git clone https://github.com/pruger-dev/my-network-labs.git /srv/containerlab-node-manager/projects/my-network-labs
```

Expected result Cloning into `'/srv/containerlab-node-manager/projects/my-network-labs'`...

If not If the lab VM refuses to write into that folder, ask your instructor to add your VM account to the lab-folder group.

Action Back in the manager, from Home click **Choose a file on the lab VM...**, open `/srv/containerlab-node-manager/projects` → `my-network-labs` → `my-first-lab` (the folder inside the clone) → click ◇ **my-first-lab.clab.yml**. In the **Topology file** dialog, check that **File location on the VM** reads `/srv/containerlab-node-manager/projects/my-network-labs/my-first-lab/my-first-lab.clab.yml` — it must contain `my-network-labs`. The original `my-first-lab` folder is still listed at the top level and looks the same; picking that one instead would deploy the old copy, not the one you just rebuilt from Git. Click **Deploy lab**, then confirm the **Start my-first-lab?** review with **Start lab**.

Expected result Both devices read **Ready**, and the Topology tab shows the same map as before — the annotations file came from Git along with the topology.

Action Open the **Progress** tab and **Connect a repository by URL** again, with the same HTTPS URL and folder `my-first-lab`.

Expected result The card simply reads "my-first-lab saves to my-network-labs > my-first-lab > latest/" again — the checkout registered in B6 is reused, with no special message.

Action Under **Saved versions**, find **Latest** and click **Apply to running lab...**. The review, titled **Replace running configuration**, lists both devices with their differences; confirm with **Replace configurations**.

Expected result Within about 12 seconds each device reports "Configuration replaced and verified against the saved desired state." On `r1`'s CLI, `ping 10.0.0.2` succeeds, and `show ip interface brief` on `r2` lists `Loopback0 10.255.0.2/32` — the lab is rebuilt entirely from what Git kept.

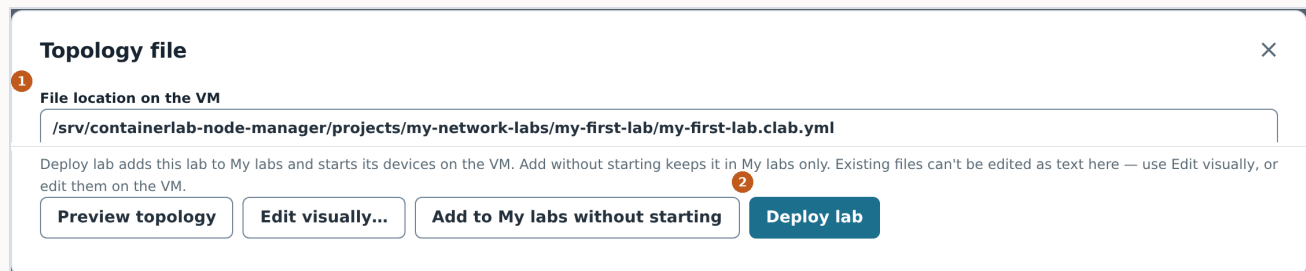


Figure B.10 — The Topology file dialog, confirming the path runs through my-network-labs.

Checklist

- [] `my-first-lab` drawn, imaged correctly and deployed
- [] Link addressed and pinging

- [] `my-network-labs` created on GitHub, private, with a first commit
- [] Repository connected, first **Save progress** uploaded
- [] Topology and map published from the lab VM; checkout clean and pushed
- [] A second change saved, and checkpoint `link-up` created
- [] Lab destroyed and removed, then redeployed from the folder inside the cloned repository
- [] Repository reconnected and **Latest** applied; the link pings and r2's loopback is back

Reference

What is saved where

What it holds	Where it lives	Created by	Survives a device restart?	Survives a Destroy lab?
A drawn but unsaved topology	This browser's storage	The lab builder, as you edit	Not applicable — no lab is deployed yet	Not applicable
The topology file and its map	A lab folder on the lab VM	Save to the VM... / Deploy lab / Write a new topology...	Yes	Yes — Destroy removes the running devices and the generated lab folder, not the original topology file
A device's running configuration	The device itself	Commands you type; <code>write memory</code> on EOS	Only if written to startup	No — Destroy removes the containers
A local progress save (status Saved on this VM — not uploaded)	The registered checkout on the lab VM	Save progress , then Not now — keep it on the VM	Yes	Yes
An uploaded Latest	Your online repository	Upload these changes / Upload now	Yes	Yes
A named checkpoint	Your online repository (and locally until uploaded)	Create checkpoint...	Yes	Yes
The result of Apply to running lab...	The running devices	Apply to running lab... , confirmed with Replace configurations	Yes — on EOS the manager runs <code>write memory</code> for you once the change is confirmed	No — Destroy removes the containers; redeploy and apply again

Stop, Destroy, Remove

Stop devices keeps the containers so you can start them again quickly; nothing configured is lost, but it is also not saved anywhere by stopping. **Destroy lab...** removes the running devices (and, on this manager, the generated lab folder with them) — configuration changes you never saved are lost, but your saved progress, checkpoints and backups all remain, since they were never stored on the devices. **Remove from this manager...** only forgets the lab in this manager; it never deletes anything on the lab VM or in your repository, and a lab it removed can always be added back by deploying its topology file again.

Troubleshooting

Symptom	Next step	Ask your instructor if...
A device stays on Starting for several minutes	Wait — cEOS and similar network operating systems take up to a couple of minutes to boot and answer a login	It never reaches Ready
A device shows Needs credentials	Ask your instructor for the login this lab expects, enter it, then click Test login now in the device panel	You do not know the intended login for this lab
A save reads Saved on this VM — not uploaded	Open it under Recent saves and click Review and upload... , then Upload these changes	The upload keeps failing after several tries
An upload or sync is rejected	Check you are connecting your own copy of the repository, not a read-only one, and that the acknowledgement was ticked	The rejection names a permission or repository problem you cannot resolve
No Apply to running lab... button, or a device is listed as skipped in the review	Read the reason shown next to that device — a platform this manager cannot restore yet, or a saved version made before this device's platform supported it	The reason is not one of these
Deploy fails naming an image the VM does not have	Open the lab with Edit visually... , correct the device's Image field, save, and deploy again	The image you were told to use still is not found

Next session in 60 seconds

- Open the manager, open your lab from Home, click **Start lab** if needed, and wait for **Ready**.
- Check the **Progress** tab for anything waiting under **Recent saves**.
- Pick up where you left off, or apply a saved version to jump to a specific point.
- End the session with **Save progress** and **Upload these changes**.